Autonomous Indoor Inventory Verification: A ROS 2 Jazzy Approach with Fiducial Marker Tracking

*Course Project: ME-580 - University of North Dakota

Omar Elalaily
Dept. of Mechanical Engineering
University of North Dakota
Grand Forks, USA
omar.elalaily@und.edu

***Abstract*—This paper presents the design and implementation of an autonomous mobile robot (AMR) system for performing automated inventory verification in a structured indoor environment. The system is built entirely on the ROS 2 Jazzy framework, utilizing the Raspberry Pi 5 as its core compute platform. The approach combines LiDAR-based Adaptive Monte Carlo Localization (AMCL) from the Nav2 stack for robust navigation with a vision-based pipeline for semantic perception. The robot navigates a predefined sequence of waypoints, pausing at inventory bays to detect ArUco markers identifying both the bay and the products. A custom auditing node applies inventory logic to generate a real-time CSV report, flagging misplaced items. The successful integration of these components demonstrates a complete solution for high-precision, low-cost autonomous cycle counting.**

***Index Terms*—ROS 2 Jazzy, Nav2, ArUco, Inventory Automation, Mobile Robotics, Perception-Action Loop.**

I. Introduction and Background

The demand for efficient, high-frequency inventory auditing drives the application of Autonomous Mobile Robots (AMRs) in logistics and retail [1]. Traditional methods of cycle counting are slow, labor-intensive, and prone to error. This project implements a small-scale AMR to autonomously patrol a mapped area and perform planogram compliance checks using fiducial markers.

The project's architecture integrates core robotics principles: reliable localization and path planning via the **Nav2** stack [4] with high-level perception and decision-making. The use of the ArUco system [2] provides a robust and computationally light method for data association in dynamic environments. Our system provides a direct, traceable audit record, connecting the robot's physical location with semantic information (Bay ID and Product ID).

II. System Description

A. Hardware and Setup

The system is built on a differential drive robot platform, leveraging high-performance and low-power components:

- **Compute Unit:** Raspberry Pi 5 running Ubuntu and ROS 2 Jazzy, employing a containerized architecture. The robot base driver and the vision subsystem run in isolated Docker containers to ensure modularity and resource isolation.
- **LiDAR:** 2D LiDAR sensor for range measurements, publishing to the system scan topic.
- **Odometry:** Wheel encoders providing local motion estimates to the AMCL node.
- **Camera:** Monocular USB Camera for ArUco marker detection.

The robot's geometric structure is defined by a custom URDF file. This file establishes the necessary transforms (`base_footprint` → `base_link` → `laser`) crucial for stable data fusion and visualization in RViz. The robot's kinematic chain is defined via a Unified Robot Description Format (URDF). A static transform publisher broadcasts the relationship between the `base_link` (geometric center) and the `laser` frame. Based on the physical mounting, the LiDAR is offset by:

$$T_{base \to laser} = [x = 0.0445, y = 0.0094, z = 0.0811] \quad (1)$$

Correctly defining this offset is critical; a misalignment of even a few centimeters in the URDF would result in a discrepancy between the laser scan and the inflation layer, causing the navigation stack to perceive clear paths as blocked.

B. Software Architecture

The software stack is organized into three main functional layers:

1) **Mapping and Localization:** The occupancy grid was generated from vector data using a custom conversion script. Localization relies on Nav2's **AMCL** node, tuned via a YAML configuration file to optimally fuse LiDAR data with odometry.
2) **Navigation Control:** The **Patrol Coordinator** node utilizes the Nav2 Simple Commander API to manage the mission. It initializes the robot's pose and dispatches sequential waypoints.
3) **Perception and Auditing:** This layer comprises the **Perception Node** (which utilizes OpenCV to publish marker IDs) and the core **Auditing Node**, which logs the final inventory status.

C. Vector-to-Raster Map Generation Approach

Unlike traditional SLAM-based mapping, which often introduces sensor noise and loop-closure artifacts into the occupancy grid, this project utilized a novel Vector-to-Raster conversion approach. The environment layout was first defined as a set of geometric primitives (line segments) in a JSON format. A custom Python script was developed to rasterize these vectors into a noise-free Portable Gray Map (PGM).

This approach offers two distinct advantages for inventory robots:

1) **Ground Truth Accuracy:** The map perfectly matches the physical dimensions of the warehouse layout, eliminating the drift often seen in Gmapping or Slam-Toolbox generated maps.
2) **Clear Boundaries:** The "occupied" pixels are deterministic, providing sharp gradients for the AMCL likelihood field model, which significantly improves localization convergence speed.

III. Methodology

A. Waypoint Navigation and Pausing Logic

The robot executes a predefined patrol route to visit the inventory bays. The waypoints are defined by (x, y) coordinates in meters and a yaw angle (ψ) in radians. The robot's initial pose and the patrol waypoints are specified in the coordinator script.

TABLE I
Navigation Waypoints

Point	X (m)	Y (m)	Yaw (ψ)	Objective
Start	7.62	0.50	π rad	Initialize at Corner
WP 1	5.08	2.54	0.0 rad	Approach Bay 1
WP 2	5.84	1.52	π rad	Approach Bay 2
WP 3	3.81	2.00	π rad	Intermediate Safety Buffer
WP 4	2.54	2.54	π rad	Approach Bay 3

The orientation is converted from Euler yaw (ψ) to a quaternion q for ROS 2 messages using the transformation:

$$q_z = \sin(\psi/2), \quad q_w = \cos(\psi/2)$$

Specifically, **Waypoint 3** was introduced as an intermediate transition point between Bay 2 and Bay 3. This safety buffer ensures the global planner generates a smooth trajectory away from the inflation radius of the narrow corridor obstacles, preventing potential collisions during the rotation maneuver.

As configured in the navigation parameters, a dedicated plugin enforces a waypoint pause duration of 5 seconds upon goal achievement, ensuring the camera has sufficient time to capture and process the ArUco markers without motion blur.

B. Navigation Architecture: Costmaps and Planning

The navigation system relies on a layered costmap implementation. We utilize two distinct costmaps:

- **Global Costmap:** A static map layer representing the permanent walls and shelves, used for long-term path planning. It includes an inflation layer where the cost $C(d)$ decays exponentially from obstacles to ensure the robot maintains a safe clearance.
- **Local Costmap:** A rolling window map centered on the robot, updated in real-time by the LiDAR. This allows the robot to react to dynamic or unmapped obstacles.

C. Path Planning Algorithms

The Nav2 stack employs a hierarchical planning architecture:

1) **Global Planner (Smac/NavFn):** We utilize a grid-based planner to compute the optimal kinematic path from the start pose to the goal pose across the global costmap. This planner ensures the shortest path while respecting the inflation costs.
2) **Local Planner (Controller):** The **Regulated Pure Pursuit (RPP)** or **DWB** controller tracks the global path. It generates velocity commands (v, ω) by simulating multiple trajectories and scoring them based on path alignment, goal distance, and obstacle clearance. This ensures smooth motion execution and obstacle avoidance.

D. Inventory Verification Logic

The **Auditing Node** implements the decision logic based on a decoupled YAML database. The logic operates on the state of the robot's last known location.

1) **State Update:** When a Bay ID (1, 2, 3) is detected, the logger's internal location state is updated.
2) **Product Check:** When a Product ID (4, 5, 6) is detected, the SKU is mapped to its **Expected Bay** using the database.
3) **Verification:** The location status is determined by comparing the detected product's *Expected Bay* with the robot's *Current Bay*.

The output is logged to a CSV file. For example, a successful misplacement alert is recorded as:

```
"11:41:22,6,PRODUCT,cereal,"WRONG
(Found in Bay1, Expected Bay3)"
```

This line serves as a quantitative metric, demonstrating the system's ability to validate data against ground truth.

E. Probabilistic Localization and Costmap Modeling

To ensure robust navigation, the system relies on the Adaptive Monte Carlo Localization (AMCL) algorithm. The belief distribution of the robot's pose x_t at time t is approximated by a set of M particles $X_t = \{x_t^{[1]}, x_t^{[2]}, ..., x_t^{[M]}\}$. The update step fuses the odometry motion model $P(x_t|u_t, x_{t-1})$ with the LiDAR measurement model $P(z_t|x_t, m)$ according to:

$$P(x_t|z_t, u_t, m) = \eta P(z_t|x_t, m) \int P(x_t|u_t, x_{t-1}) P(x_{t-1}) dx_{t-1} \tag{2}$$

Where η is a normalization constant and m is the occupancy grid map.

Furthermore, safety is enforced via the global costmap inflation layer. The cost values $C(x, y)$ in the configuration space decrease exponentially with distance d from the nearest obstacle, defined as:

$$C(d) = \begin{cases} 254 & \text{if } d \leq r_{inscribed} \\ 253 \cdot e^{-\alpha(d - r_{inscribed})} & \text{if } d > r_{inscribed} \end{cases} \quad (3)$$

This decay function ensures that the path planner naturally generates trajectories that maximize clearance from shelf corners, critical for the narrow aisle environment.

IV. Results

A. Navigation Performance

The Nav2 stack, utilizing the clean, converted map and AMCL, provided highly accurate localization. Experimental trials confirmed that the global planner successfully generated paths that minimized travel distance while respecting the inflation layer defined in the costmap.

B. Auditing Output

The final output of the system is the CSV log. This log serves as the end-to-end audit report.

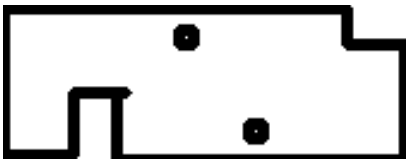


Fig. 1. The occupancy grid map. This map was generated by a custom conversion script from vector data, ensuring high precision for AMCL. The path shown in the RViz visualization confirms the robot's precise movement.



Fig. 2. Robot at Waypoint 1 (Bay 1). The camera detects ID 1 (Bay) and IDs 4 & 6 (Products). CSV Interpretation (Time $\sim$ 11:41:22): The log correctly identifies Silverware (ID 4) as CORRECT for Bay 1, but flags Cereal (ID 6) as WRONG, reporting "Expected Bay 3."



Fig. 3. Robot at Waypoint 3 (Bay 2). The robot localizes at the second aisle. CSV Interpretation: The system detects ID 5 (Pasta). Consulting the database, it confirms that Pasta is assigned to Bay 2, resulting in a CORRECT status log. while flagging "Silverware" as MISPLACED (since its designated location is Bay 1), and "Cereal" as MISPLACED (since its designated location is Bay 3).

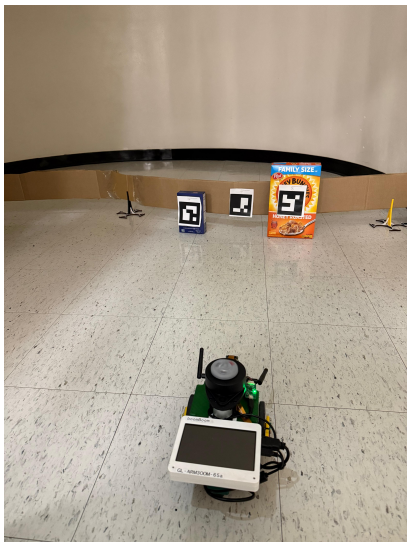


Fig. 4. Robot at Waypoint 4 (Bay 3). The current logic state is updated to Bay 3. CSV Interpretation: The system successfully identified "Cereal" as (assigned to Bay 3), while flagging "Pasta" as MISPLACED (since its designated location is Bay 2).

V. Discussion

The integration of a layered costmap architecture was crucial for the safe navigation of the robot in the confined aisles. By carefully tuning the inflation radius in the global costmap, we ensured that the planner generated paths that kept the robot centrally aligned in the corridors, preventing side-collisions. Furthermore, the decoupling of the navigation stack (Nav2) from the perception logic (ArUco Logger) proved to be a robust architectural decision. This modularity allowed

for independent tuning of the navigation parameters without disrupting the auditing logic.

However, some limitations were observed. The AMCL algorithm relies heavily on distinct geometric features. In highly symmetric environments (like long, identical warehouse aisles), the particle filter can suffer from ”kidnapped robot” problems. Future implementations could benefit from fusing visual odometry or utilizing WiFi signal strength as an additional observation source to disambiguate the robot’s pose.

VI. Conclusion and Future Work

The project successfully delivered a fully functional autonomous inventory verification system on a cost-effective Raspberry Pi 5 platform using ROS 2 Jazzy. The successful integration of Nav2 for sequential waypoint navigation, ArUco for semantic perception, and a custom logic layer for misplacement detection demonstrates a robust solution to the indoor auditing problem. The final CSV report provides clear, auditable evidence of inventory compliance.

A. Possible Improvements

- **Dynamic Obstacle Avoidance:** The current system uses only static map features. Integrating a dynamic obstacle detector into the coordinator would allow the robot to handle unexpected floor clutter (like fallen boxes or people) more gracefully.
- **3D Perception:** Using a depth camera (like an Intel RealSense) could allow the robot to estimate the marker pose, providing more precise location data within the bay (e.g., top shelf vs. bottom shelf).
- **EKF Fusion:** While AMCL is used, implementing an Extended Kalman Filter (EKF) or complementary filter with IMU data would improve odometry fusion and resilience to wheel slip [1].

References

[1] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, 2nd ed. Cambridge, MA: MIT Press, 2011.

[2] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, and M. J. Marín-Jiménez, ”Automatic generation and detection of highly reliable fiducial markers under occlusion,” *Pattern Recognition*, vol. 47, no. 6, pp. 2280-2292, 2014.

[3] G. Popović, I. Cvišić, I. Marković, and I. Petrović, ”Human localization in warehouse environments based on a wearable camera sensor suite and dynamic ultra-wideband sensing,” in *Proc. 20th Int. Conf. on Advanced Robotics (ICAR)*, 2021, pp. 1-7.

[4] S. Macenski, F. Martín, R. White, and J. Clavero, ”The Marathon 2: A Navigation System,” in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Las Vegas, NV, USA, 2020, pp. 2518-2525.